Мухаметшин А.Р. ББМО-01-25

```
# %matplotlib inline
# подключаем библиотеки
import os
import random
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from matplotlib.pyplot import imshow
from tensorflow.keras.preprocessing import image # преодобработка изображени
from tensorflow.keras.applications.imagenet_utils import preprocess_input #
from tensorflow.keras.models import Sequential # последовательное соединение
from tensorflow.keras.layers import Dense, Dropout, Flatten, Activation # сл
from tensorflow.keras.layers import Conv2D, MaxPooling2D # слои, которые буд
from tensorflow.keras.models import Model # объект нейронной сети в который
```

```
import gdown # модуль для загрузки с гугл-диска
url = 'https://drive.google.com/uc?id=1Madin3qXe3Xexox5qOhASGulFR2wvsFz' # а
output = '101_ObjectCategories.tar.gz' # название загружаемого файла
gdown.download(url, output, quiet=False) # загружаем
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1Madin3qXe3Xexox5qOhASGulFR2w
From (redirected): https://drive.google.com/uc?id=1Madin3qXe3Xexox5qOhASGulFR
To: /content/101_ObjectCategories.tar.gz
100%|██████████| 132M/132M [00:03<00:00, 37.9MB/s]
'101_ObjectCategories.tar.gz'
```

```
!echo "Downloading 101_Object_Categories for image notebooks"
# распаковываем
!tar -xzf 101_ObjectCategories.tar.gz
# удаляем после распаковки
!rm 101_ObjectCategories.tar.gz
!ls
```

```
Downloading 101_Object_Categories for image notebooks
101_ObjectCategories  sample_data
```

```
root = '101_ObjectCategories' # директория с файлами
exclude = ['BACKGROUND_Google', 'Motorbikes', 'airplanes', 'Faces_easy', 'Fa
train_split, val_split = 0.7, 0.15 # доли данных для обучения и проверки
# проходим по всем поддиректориям, они и есть наши классы
categories = [x[0] for x in os.walk(root) if x[0]][1:]
# но исключим некоторые из них
categories = [c for c in categories if c not in [os.path.join(root, e) for e
# посмотрим на классы. Специально оставили в названиях и общее название набо
print(categories)
```

```
['101_ObjectCategories/stop_sign', '101_ObjectCategories/grand_piano', '101_O
```

```
def get_image(path):
  img = image.load_img(path, target_size=(224, 224)) # загружаем изображение
  x = image.img_to_array(img) # конвертируем в массив
  x = np.expand_dims(x, axis=0) # добавляем измерение (batch) 1*224*224*3
  x = preprocess_input(x) # предобработка
  return img, x # возвращаем изображение PIL и массив для него
```

```
data = [] # пустой массив для данных
for c, category in enumerate(categories): # для всех классов, они же поддире
  # берем названия всех файлов в ней, объединяем с названием (путем к) самой
  # но только такие, в которых расширение '.jpg','.png','.jpeg'.
  images = [os.path.join(dp, f) for dp, dn, filenames
    in os.walk(category) for f in filenames
    if os.path.splitext(f)[1].lower() in ['.jpg','.png','.jpeg']]
  # для всех изображений класса
  for img_path in images:
    img, x = get_image(img_path) # загружаем его
    data.append({'x':np.array(x[0]), 'y':c}) # добавляем в словарь 'x' - мас
# число классов
num_classes = len(categories)
random.shuffle(data) # перемешаем случайно
import sys
def sizeof_fmt(num, suffix='B'):
  ''' by Fred Cirera, https://stackoverflow.com/a/1094933/1870254, modified'
  for unit in ['','Ki','Mi','Gi','Ti','Pi','Ei','Zi']:
    if abs(num) < 1024.0:
      return "%3.1f %s%s" % (num, unit, suffix)
    num /= 1024.0
  return "%.1f %s%s" % (num, 'Yi', suffix)
```

```
N=5# часть данных для экономии
idx_val = int(train_split * len(data)/N) # индекс начала для проверочно
idx_test = int((train_split + val_split) * len(data)/N) # индекс начала
idx_end=int(int( 1 * len(data)/N))
# разделяем
train = data[:idx_val] #
val = data[idx_val:idx_test] #
test = data[idx_test:idx_end] #
# разделяем данные и метки в отдельные переменные
x_train, y_train = np.array([t["x"] for t in train]), np.array([t["y"]
```

```
x_val, y_val = np.array([t["x"] for t in val]), np.array([t["y"] for t :
x_test, y_test = np.array([t["x"] for t in test]), np.array([t["y"] for
#print(y_test)
# Память
for name, size in sorted(((name, sys.getsizeof(value)) for name, value :
  key= lambda x: -x[1])[:20]:
  print("{:>30}: {:>8}".format(name, sizeof_fmt(size)))
```

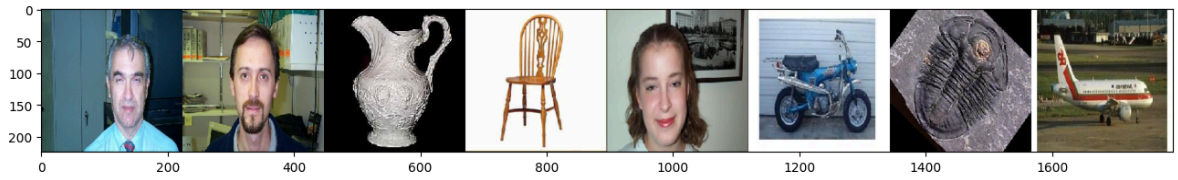
```
                      x_train: 499.0 MiB
                       x_test: 106.8 MiB
                        x_val: 106.8 MiB
                           _6: 58.7 KiB
                         data: 51.8 KiB
                      y_train:  6.9 KiB
                        train:  6.8 KiB
                          _i9:  2.4 KiB
                           _i:  2.1 KiB
                         _i20:  2.1 KiB
                         _i21:  1.7 KiB
                   Sequential:  1.7 KiB
                        Dense:  1.7 KiB
                      Dropout:  1.7 KiB
                      Flatten:  1.7 KiB
                   Activation:  1.7 KiB
                       Conv2D:  1.7 KiB
                 MaxPooling2D:  1.7 KiB
                        Model:  1.7 KiB
                       y_test:  1.6 KiB
```

```
# нормализация
x_train = x_train.astype('float32') / 255.
x_val = x_val.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
# НЕНАДО для sparse cross entropy
# конвертируем номера классов в one-hot вектора
#y_train = tf.keras.utils.to_categorical(y_train, num_classes)
#y_val = tf.keras.utils.to_categorical(y_val, num_classes)
#y_test = tf.keras.utils.to_categorical(y_test, num_classes)
#print(y_test.shape)
# ЕСЛИ sparse cross entropy
y_train = np.array(y_train)
y_val = np.array(y_val)
y_test = np.array(y_test)
# summary
print("finished loading %d images from %d categories"%(len(data), num_classe
print("train / validation / test split: %d, %d, %d"%(len(x_train), len(x_val
print("training data shape: ", x_train.shape)
print("training labels shape: ", y_train.shape)
```

```
finished loading 6209 images from 97 categories
train / validation / test split: 869, 186, 186
training data shape:  (869, 224, 224, 3)
training labels shape:  (869,)
```

```
# названия всех изображений
images = [os.path.join(dp, f) for dp, dn, filenames in os.walk(root) for f i
# выбираем случайно 8 штук
idx = [int(len(images) * random.random()) for i in range(8)]
# загружаем эти изображения
imgs = [image.load_img(images[i], target_size=(224, 224)) for i in idx]
# объединяем в одну строку
concat_image = np.concatenate([np.asarray(img) for img in imgs], axis=1)
# рисуем
plt.figure(figsize=(16,4))
plt.imshow(concat_image)
```

```
<matplotlib.image.AxesImage at 0x7d4550243dd0>
```




```
# создаем нейронную сеть
channels=16
model = Sequential() # последовательное подключение слоев
print("Input dimensions: ",x_train.shape[1:]) # размер входа
model.add(Conv2D(channels, (3, 3), input_shape=x_train.shape[1:])) # св
model.add(Activation('relu')) # активация
model.add(MaxPooling2D(pool_size=(2, 2))) # пулинг
model.add(Conv2D(channels, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25)) # dropout
model.add(Conv2D(channels, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(channels, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Flatten()) # вытягиваем результаты сверток в вектор
model.add(Dense(128))# полносвязный слой
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes)) # полносвязный слой классификации
model.add(Activation('softmax')) # активация softmax
model.summary() # напечатаем информацию о сети
```

```
Input dimensions:  (224, 224, 3)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

**Model: "sequential_1"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d_2 (Conv2D) | (None, 222, 222, 16) | 448 |
| activation (Activation) | (None, 222, 222, 16) | 0 |
| max_pooling2d_2 (MaxPooling2D) | (None, 111, 111, 16) | 0 |
| conv2d_3 (Conv2D) | (None, 109, 109, 16) | 2,320 |
| activation_1 (Activation) | (None, 109, 109, 16) | 0 |
| max_pooling2d_3 (MaxPooling2D) | (None, 54, 54, 16) | 0 |
| dropout (Dropout) | (None, 54, 54, 16) | 0 |
| conv2d_4 (Conv2D) | (None, 52, 52, 16) | 2,320 |
| activation_2 (Activation) | (None, 52, 52, 16) | 0 |
| max_pooling2d_4 (MaxPooling2D) | (None, 26, 26, 16) | 0 |
| conv2d_5 (Conv2D) | (None, 24, 24, 16) | 2,320 |
| activation_3 (Activation) | (None, 24, 24, 16) | 0 |
| max_pooling2d_5 (MaxPooling2D) | (None, 12, 12, 16) | 0 |
| dropout_1 (Dropout) | (None, 12, 12, 16) | 0 |
| flatten_1 (Flatten) | (None, 2304) | 0 |
| dense_1 (Dense) | (None, 128) | 295,040 |
| activation_4 (Activation) | (None, 128) | 0 |

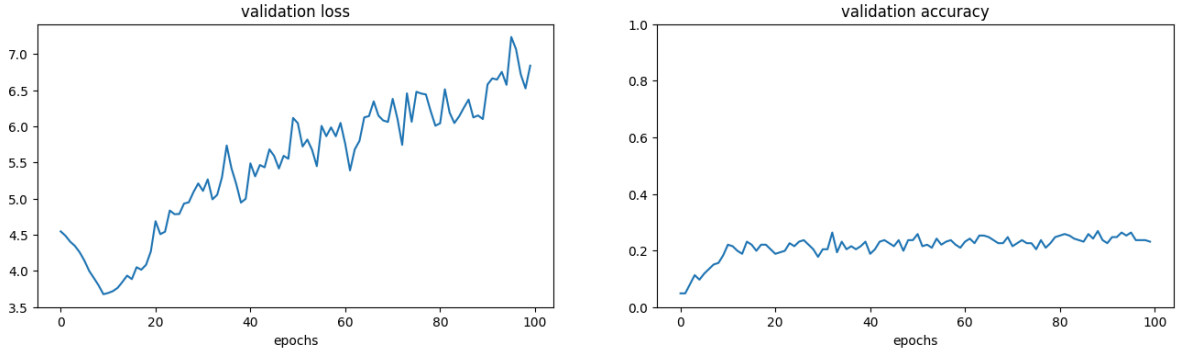
```
model.compile(loss='SparseCategoricalCrossentropy', # функция ошибки
  optimizer='adam', # метод обучения
  metrics=['accuracy']) # измеряемые метрики
# обучаем модель
history = model.fit(x_train, # примеры входа
  y_train, # указания учителя
  batch_size=32, # размер пакета
  epochs=100, # количество эпох
  validation_data=(x_val, y_val)) # данные для проверки
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.9333 - loss: 0.2155 -
Epoch 77/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9226 - loss: 0.2524 -
Epoch 78/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9439 - loss: 0.1889 -
Epoch 79/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9226 - loss: 0.2136 -
Epoch 80/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9295 - loss: 0.2007 -
Epoch 81/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9358 - loss: 0.1990 -
Epoch 82/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9346 - loss: 0.2305 -
Epoch 83/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9453 - loss: 0.1857 -
Epoch 84/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9318 - loss: 0.1796 -
Epoch 85/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9347 - loss: 0.2099 -
Epoch 86/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 36ms/step - accuracy: 0.9466 - loss: 0.1989 -
Epoch 87/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9487 - loss: 0.1875 -
Epoch 88/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9386 - loss: 0.2106 -
Epoch 89/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9342 - loss: 0.2092 -
Epoch 90/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9391 - loss: 0.2062 -
Epoch 91/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9353 - loss: 0.2165 -
Epoch 92/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9598 - loss: 0.1414 -
Epoch 93/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9541 - loss: 0.1475 -
Epoch 94/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9417 - loss: 0.1712 -
Epoch 95/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - accuracy: 0.9649 - loss: 0.1398 -
Epoch 96/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9578 - loss: 0.1429 -
Epoch 97/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9644 - loss: 0.1418 -
Epoch 98/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9401 - loss: 0.1908 -
Epoch 99/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9232 - loss: 0.2621 -
Epoch 100/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - accuracy: 0.9514 - loss: 0.1502 -
```

```
fig = plt.figure(figsize=(16,4))
ax = fig.add_subplot(121)
ax.plot(history.history["val_loss"]) # функция ошибки
ax.set_title("validation loss")
ax.set_xlabel("epochs")
ax2 = fig.add_subplot(122)
ax2.plot(history.history["val_accuracy"]) # метрика аккуратности
ax2.set_title("validation accuracy")
```

```
ax2.set_xlabel("epochs")
ax2.set_ylim(0, 1)
```

```
(0.0, 1.0)
```




```
for name, size in sorted(((name, sys.getsizeof(value)) for name, value in lo
  key= lambda x: -x[1])[:20]:
  print("{:>30}: {:>8}".format(name, sizeof_fmt(size)))
```

```
                      x_train: 499.0 MiB
                       x_test: 106.8 MiB
                        x_val: 106.8 MiB
                 concat_image:  1.1 MiB
                       images: 73.9 KiB
                           _6: 58.7 KiB
                         data: 51.8 KiB
                      y_train:  6.9 KiB
                        train:  6.8 KiB
                          _i9:  2.4 KiB
                         _i20:  2.1 KiB
                         _iii:  2.1 KiB
                         _i25:  2.1 KiB
                         _i21:  1.7 KiB
                   Sequential:  1.7 KiB
                        Dense:  1.7 KiB
                      Dropout:  1.7 KiB
                      Flatten:  1.7 KiB
                   Activation:  1.7 KiB
                       Conv2D:  1.7 KiB
```

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=0) # проверяем на те
print('Test loss:', loss)
print('Test accuracy:', accuracy)
```

```
Test loss: 7.778049468994141
Test accuracy: 0.24193547666072845
```

```
# скачиваем обученную сеть
from tensorflow.keras import applications
vgg = applications.VGG16(weights='imagenet', include_top=True)
vgg.summary() # ее описание
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applica
553467096/553467096 ━━━━━━━━━━━━━━━━━━━━ 5s 0us/step
Model: "vgg16"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| input_layer_2 (InputLayer) | (None, 224, 224, 3) | 0 |
| block1_conv1 (Conv2D) | (None, 224, 224, 64) | 1,792 |
| block1_conv2 (Conv2D) | (None, 224, 224, 64) | 36,928 |
| block1_pool (MaxPooling2D) | (None, 112, 112, 64) | 0 |
| block2_conv1 (Conv2D) | (None, 112, 112, 128) | 73,856 |
| block2_conv2 (Conv2D) | (None, 112, 112, 128) | 147,584 |
| block2_pool (MaxPooling2D) | (None, 56, 56, 128) | 0 |
| block3_conv1 (Conv2D) | (None, 56, 56, 256) | 295,168 |
| block3_conv2 (Conv2D) | (None, 56, 56, 256) | 590,080 |
| block3_conv3 (Conv2D) | (None, 56, 56, 256) | 590,080 |
| block3_pool (MaxPooling2D) | (None, 28, 28, 256) | 0 |
| block4_conv1 (Conv2D) | (None, 28, 28, 512) | 1,180,160 |
| block4_conv2 (Conv2D) | (None, 28, 28, 512) | 2,359,808 |
| block4_conv3 (Conv2D) | (None, 28, 28, 512) | 2,359,808 |
| block4_pool (MaxPooling2D) | (None, 14, 14, 512) | 0 |
| block5_conv1 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_conv2 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_conv3 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_pool (MaxPooling2D) | (None, 7, 7, 512) | 0 |
| flatten (Flatten) | (None, 25088) | 0 |
| fc1 (Dense) | (None, 4096) | 102,764,544 |
| fc2 (Dense) | (None, 4096) | 16,781,312 |
| predictions (Dense) | (None, 1000) | 4,097,000 |

```
# ссылка на вход VGG
inp = vgg.input
# новый полносвязный слой классификации с softmax и num_classes нейронов
new_classification_layer = Dense(num_classes, activation='softmax')
# подключаем новый слой к выходу предпоследнего слоя VGG (индекс -2), и дела
out = new_classification_layer(vgg.layers[-2].output)
# создаем сеть от входа до нового выхода.
```

```
model_new = Model(inp, out)
```

```
# запрещаем обучение всех весов\смещений для слоев кроме последнего
for l, layer in enumerate(model_new.layers[:-1]):
  layer.trainable = False
# на всякий случай последнему слою разрешаем обучаться
for l, layer in enumerate(model_new.layers[-1:]):
  layer.trainable = True
# компилируем модель и указываем опции обучения
model_new.compile(loss='SparseCategoricalCrossentropy',
optimizer='adam',
metrics=['accuracy'])
model_new.summary() # печатаем информацию о модели
```

```
Model: "functional_21"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| input_layer_2 (InputLayer) | (None, 224, 224, 3) | 0 |
| block1_conv1 (Conv2D) | (None, 224, 224, 64) | 1,792 |
| block2_conv2 (Conv2D) | (None, 112, 112, 128) | 147,584 |
| block2_pool (MaxPooling2D) | (None, 56, 56, 128) | 0 |
| block3_conv1 (Conv2D) | (None, 56, 56, 256) | 295,168 |
| block3_conv2 (Conv2D) | (None, 56, 56, 256) | 590,080 |
| block3_conv3 (Conv2D) | (None, 56, 56, 256) | 590,080 |
| block3_pool (MaxPooling2D) | (None, 28, 28, 256) | 0 |
| block4_conv1 (Conv2D) | (None, 28, 28, 512) | 1,180,160 |
| block4_conv2 (Conv2D) | (None, 28, 28, 512) | 2,359,808 |
| block4_conv3 (Conv2D) | (None, 28, 28, 512) | 2,359,808 |
| block4_pool (MaxPooling2D) | (None, 14, 14, 512) | 0 |
| block5_conv1 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_conv2 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_conv3 (Conv2D) | (None, 14, 14, 512) | 2,359,808 |
| block5_pool (MaxPooling2D) | (None, 7, 7, 512) | 0 |
| flatten (Flatten) | (None, 25088) | 0 |
| fc1 (Dense) | (None, 4096) | 102,764,544 |
| fc2 (Dense) | (None, 4096) | 16,781,312 |
| dense_3 (Dense) | (None, 97) | 397,409 |

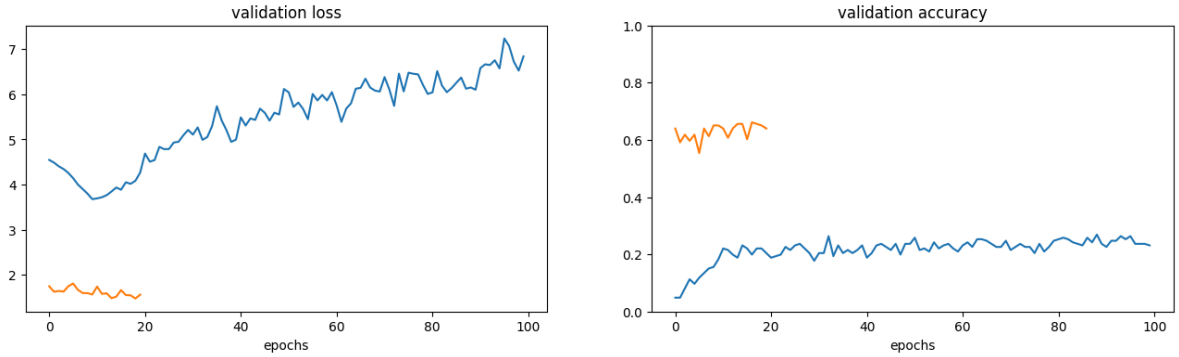
**Total params:** 134,657,953 (513.68 MB)

```
# обучаем
history2 = model_new.fit(x_train, y_train,
batch_size=32,
epochs=20,
validation_data=(x_val, y_val))
```

```
Epoch 1/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 227ms/step - accuracy: 0.9343 - loss: 0.4087 -
Epoch 2/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 222ms/step - accuracy: 0.9439 - loss: 0.3639 -
Epoch 3/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 226ms/step - accuracy: 0.9180 - loss: 0.4250 -
Epoch 4/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 7s 234ms/step - accuracy: 0.9089 - loss: 0.4094 -
Epoch 5/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 7s 235ms/step - accuracy: 0.9222 - loss: 0.3717 -
Epoch 6/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 231ms/step - accuracy: 0.9292 - loss: 0.3224 -
Epoch 7/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 223ms/step - accuracy: 0.9502 - loss: 0.2821 -
Epoch 8/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 10s 215ms/step - accuracy: 0.9473 - loss: 0.2892 -
Epoch 9/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 218ms/step - accuracy: 0.9658 - loss: 0.2501 -
Epoch 10/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 215ms/step - accuracy: 0.9549 - loss: 0.2763 -
Epoch 11/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 217ms/step - accuracy: 0.9696 - loss: 0.2211 -
Epoch 12/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 219ms/step - accuracy: 0.9759 - loss: 0.2258 -
Epoch 13/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 221ms/step - accuracy: 0.9826 - loss: 0.1988 -
Epoch 14/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 220ms/step - accuracy: 0.9728 - loss: 0.1958 -
Epoch 15/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 223ms/step - accuracy: 0.9739 - loss: 0.2029 -
Epoch 16/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 223ms/step - accuracy: 0.9776 - loss: 0.1658 -
Epoch 17/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 10s 219ms/step - accuracy: 0.9822 - loss: 0.1462 -
Epoch 18/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 222ms/step - accuracy: 0.9915 - loss: 0.1371 -
Epoch 19/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 220ms/step - accuracy: 0.9841 - loss: 0.1508 -
Epoch 20/20
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 222ms/step - accuracy: 0.9753 - loss: 0.1538 -
```

```
fig = plt.figure(figsize=(16,4))
ax = fig.add_subplot(121)
ax.plot(history.history["val_loss"]) # функция ошибки предыдущей сети
```

```
ax.plot(history2.history["val_loss"]) # функция ошибки этой сети
ax.set_title("validation loss")
ax.set_xlabel("epochs")
ax2 = fig.add_subplot(122)
ax2.plot(history.history["val_accuracy"]) # аккуратность предыдущей сети
ax2.plot(history2.history["val_accuracy"]) # аккуратность этой сети
ax2.set_title("validation accuracy")
ax2.set_xlabel("epochs")
ax2.set_ylim(0, 1)
plt.show()
```




```
loss, accuracy = model_new.evaluate(x_test, y_test, verbose=0)
print('Test loss:', loss)
print('Test accuracy:', accuracy)
```

```
Test loss: 1.6698493957519531
Test accuracy: 0.5752688050270081
```

```
img, x = get_image('101_ObjectCategories/airplanes/image_0007.jpg')
probabilities = model_new.predict([x])
id=np.argmax(probabilities)
print(categories[id],probabilities[0,id])
img
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
101_ObjectCategories/chair 0.9461871
```